

Assignment-1

NAME: Sameera Anjum

ROLL NO: 2303A52466

COURSE: AI Assisted Coding

Question 1: Zero-Shot Prompting (Palindrome Number Program) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

PROMPT

```
1 # write a python program to check whether the given number is palindrome or not
2 def is_palindrome(number):
3     # Convert the number to string
4     str_num = str(number)
5     # Check if the string is equal to its reverse
6     return str_num == str_num[::-1]
7 # Test the function
8 num = int(input("Enter a number: "))
9 if is_palindrome(num):
10     print(f"{num} is a palindrome.")
11 else:
12     print(f"{num} is not a palindrome.")
```

OUTPUT

```
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe c:/Users/samee/OneDrive/Desktop/academics/3-2/AIAC/day2.py
Enter a number: 12021
12021 is a palindrome.
PS C:\Users\samee> |
```

ANALYSIS

- The zero-shot prompt gave a correct program for checking palindrome numbers.
- The program works well for normal positive numbers.
- It does not clearly handle negative numbers or wrong input types.
- This shows that the code works, but some special cases are missing.

Question 2: One-Shot Prompting (Factorial Calculation) Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

PROMPT

```
C: > Users > samee > OneDrive > Desktop > academics > 3-2 > AIAC > day2.py > ...  
14  
15 # write a python function to compute the factorial of a number  
16 def factorial(n):  
17     if n < 0:  
18         return "Factorial is not defined for negative numbers."  
19     elif n == 0 or n == 1:  
20         return 1  
21     else:  
22         result = 1  
23         for i in range(2, n + 1):  
24             result *= i  
25         return result  
26 num = int(input("Enter a number to compute its factorial: "))  
27 print(f"The factorial of {num} is {factorial(num)}")  
28
```

OUTPUT

```
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe c:/Users/samee/OneDrive/Desktop/academics/3-2/AIAC/day2.py  
Enter a number to compute its factorial: 5  
The factorial of 5 is 120  
PS C:\Users\samee> █
```

ANALYSIS

- One-shot prompting produces better and safer code than zero-shot prompting.
- The example helps the AI write clearer logic and handle edge cases correctly.

Question 3: Few-Shot Prompting (Armstrong Number Check) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.

- Test the function with boundary values and invalid inputs.

PROMPT

```
31 def is_armstrong(number):
32     # Calculate the number of digits
33     num_digits = 0
34     temp = number
35     while temp > 0:
36         temp //= 10
37         num_digits += 1
38     # Calculate the sum of the digits raised to the power of num_digits
39     sum_of_powers = 0
40     temp = number
41     while temp > 0:
42         digit = temp % 10
43         sum_of_powers += digit ** num_digits
44         temp //= 10
45     # Check if the sum of powers is equal to the original number
46     return sum_of_powers == number
47 # Test the function
48 num = int(input("Enter a number: "))
49 if is_armstrong(num):
50     print(f"{num} is an Armstrong number.")
51 else:
52     print(f"{num} is not an Armstrong number.")
```

OUTPUT

```
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe c:/Users/samee/OneDrive/Desktop/academics/3-2/AIAC/day2.py
Enter a number: 153
153 is an Armstrong number.
PS C:\Users\samee> |
```

ANALYSIS

- The code structure is better and follows the examples.
- The code does not handle negative or invalid inputs properly.
- Accuracy is higher compared to zero-shot and one-shot prompting.

Question 4: Context-Managed Prompting (Optimized Number Classification) Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.
- Compare the output with earlier prompting strategies.

PROMPT

```

53
54 # give an optimized python code that classifies the given number as composite or prime or neither
55 def classify_number(n):
56     if n <= 1:
57         return "neither prime nor composite"
58     elif n == 2:
59         return "prime"
60     elif n % 2 == 0:
61         return "composite"
62     else:
63         for i in range(3, int(n**0.5) + 1, 2):
64             if n % i == 0:
65                 return "composite"
66         return "prime"
67 num = int(input("Enter a number: "))
68 classification = classify_number(num)
69 print(f"{num} is {classification}.")
70

```

OUTPUT

```

PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe c:/Users/samee/On
Enter a number: 250
250 is composite.
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe c:/Users/samee/On
Enter a number: 57
57 is composite.
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe c:/Users/samee/On
Enter a number: 13
13 is prime.
PS C:\Users\samee>

```

ANALYSIS

- Input validation is properly handled.
- Compared to earlier prompting methods, this gives the best accuracy and performance.
- Clear instructions helped the AI write correct and efficient code.

Question 5: Zero-Shot Prompting (Perfect Number Check) Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

PROMPT

```

71
72 # generate a python code that checks whether the given number is perfect number or not
73 def is_perfect_number(n):
74     if n <= 1:
75         return False
76     sum_of_divisors = 1 # 1 is a proper divisor
77     for i in range(2, int(n**0.5) + 1):
78         if n % i == 0:
79             sum_of_divisors += i
80             if i != n // i:
81                 sum_of_divisors += n // i
82     return sum_of_divisors == n
83 num = int(input("Enter a number: "))
84 if is_perfect_number(num):
85     print(f"{num} is a perfect number.")
86 else:
87     print(f"{num} is not a perfect number.")

```

OUTPUT

```

PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe
Enter a number: 28
28 is a perfect number.
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe
Enter a number: 120
120 is not a perfect number.
PS C:\Users\samee> 

```

ANALYSIS

The zero-shot prompt created a correct program for checking perfect numbers.

The program works well for normal positive numbers like 6 and 28.

It does not handle 0, negative numbers, or wrong inputs properly.

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation) Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even
- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

PROMPT

```
89 # generate a python code that determines whether a number is even or odd including proper input handling
90 def is_even_or_odd():
91     try:
92         num = int(input("Enter an integer: "))
93         if num % 2 == 0:
94             print(f"{num} is even.")
95         else:
96             print(f"{num} is odd.")
97     except ValueError:
98         print("Invalid input. Please enter a valid integer.")
99 is_even_or_odd()
```

OUTPUT

```
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe
Enter an integer: 120
120 is even.
PS C:\Users\samee> & C:/Users/samee/AppData/Local/Programs/Python/Python313/python.exe
Enter an integer: 123
123 is odd.
PS C:\Users\samee> |
```

ANALYSIS

The given examples helped the AI understand what even and odd mean.

The program gives clear output like “Even” or “Odd”.

It correctly works for negative numbers also.

It safely rejects non-integer inputs like decimals or strings.